MEMBER 3 • NEW `NETWORK` PACKAGE, INTERFACES & SOCKET LIBRARIES

CricBid Review 2 Guide (From Scratch)

Aligned 100% with the Review 2 Marking Rubric: Packages & Interfaces (5M) + Java Libraries & Exception Handling (5M) + Presentation (5M).

✅ YOUR REVIEW 2 RUBRIC CHECKLIST (100% NEWLY BUILT FOR REVIEW 2!)

- 📦 **Packages:** Brand-new `com.cricbid.network` + `com.cricbid.observer`
- 🔌 **Interfaces:** `AuctionObserver`, `Runnable`, `Serializable`
- 📚 **Java Libraries:** `java.net.*` (`ServerSocket`, `Socket`), `java.io.*` (`ObjectOutputStream`), `CopyOnWriteArrayList`
- 🛡️ **Exception Handling:** Network `CricBidException` routing (`BID_REJECTED`), `EOFException`, 2s retry `try-catch`

1. Your Updated 2-Minute Speech (Rubric-Aligned)

"I'll explain our brand-new `com.cricbid.network` package built for Review 2, and how it combines interfaces, Java networking libraries, and network exception handling for real-time multi-laptop play.

In Review 1, live timers and `AuctionObserver` callbacks existed only in one laptop's RAM. To synchronize three laptops without database lag, we created the `com.cricbid.network` package using **Java's `java.net` and `java.io` libraries**—specifically `ServerSocket`, `Socket`, `ObjectOutputStream`, and `ObjectInputStream` on TCP port `5050`.

For **interfaces**, `AuctionServer` implements our `AuctionObserver` interface, while each per-laptop `ClientHandler` implements `java.lang.Runnable` inside a thread-safe `java.util.concurrent.CopyOnWriteArrayList`, and `NetworkMessage` implements `Serializable`. Whenever a timer tick, bid, or sale occurs on Laptop 1, `AuctionServer` automatically broadcasts it to all connected laptops in milliseconds.

For **exception handling**, if a remote laptop's bid violates a rule, `handleRemoteBid()` catches `CricBidException` and `SQLException` and sends a targeted `BID_REJECTED` message back to only that client. On Laptops 2 and 3, `AuctionClient` catches `IOException` and `EOFException` inside a retry loop—automatically reconnecting every 2 seconds if the Host isn't up yet, applying a `SYNC_STATE` snapshot on join, and dispatching UI updates safely via `SwingUtilities.invokeLater()`."

2. Basic Java Dictionary (Explained From Zero!)

Java Rubric Term	Simple Everyday Meaning
`java.net` Library (`ServerSocket`, `Socket`)	Java's official networking library. `ServerSocket` opens port `5050` on Laptop 1; `Socket` connects Laptops 2 & 3 over Wi-Fi with zero lost packets (TCP).
`java.io` Object Streams	`ObjectOutputStream` & `ObjectInputStream` send and receive whole `NetworkMessage` Java objects across Wi-Fi instead of raw text strings.
`AuctionObserver` Interface	Our custom observer interface (8 event methods). Because `AuctionServer implements AuctionObserver`, it gets notified automatically on every tick, bid, or sale!

`Runnable` Interface	Java's built-in multithreading interface (`public void run()`). Giving each laptop its own `ClientHandler implements Runnable` lets all laptops bid simultaneously!
`CopyOnWriteArrayList` Library	A thread-safe `List` from `java.util.concurrent`. Lets Laptop 1 broadcast timer ticks even while another laptop connects or disconnects at the same instant!
`EOFException` & `IOException`	Java I/O exceptions thrown when a laptop closes its window (`EOFException`) or Wi-Fi drops (`IOException`), caught cleanly so the app never crashes.

3. File-by-File Code Walkthrough (What to Open & Point At)

File 1: `AuctionServer.java` — New Package, Interfaces & `java.net` / `concurrent` Libraries

📂 Open: src/main/java/com/cricbid/network/AuctionServer.java (Lines 1–31, 52–83, 194)

```
package com.cricbid.network; // Brand-new package added in Review 2!

import java.io.ObjectInputStream;                    // java.io library
import java.io.ObjectOutputStream;
import java.net.ServerSocket;                        // java.net library
import java.net.Socket;
import java.util.concurrent.CopyOnWriteArrayList;    // java.util.concurrent library

public class AuctionServer implements AuctionObserver { // Implements custom interface!
    private final List<ClientHandler> clients = new CopyOnWriteArrayList<>();
    ...
    public void start() throws IOException {
        serverSocket = new ServerSocket();
        serverSocket.bind(new java.net.InetSocketAddress(port)); // Port 5050
        auctionService.getAuctionSubject().addObserver(this);    // Register observer!
        Thread acceptThread = new Thread(this::acceptLoop, "CricBid-SocketServer-Accept");
        acceptThread.setDaemon(true);
        acceptThread.start();
    }
    ...
    private class ClientHandler implements Runnable { // Implements Runnable interface!
```

- **Point out how clean this design is:** `AuctionServer` implements `AuctionObserver` (lines 136–190), so every `onTimerTick`, `onBidPlaced`, and `onPlayerSold` automatically wraps the event into a `NetworkMessage` and calls `broadcast(...)`.
- **`out.reset()` inside `send()` (Lines 312–323):** Clears `ObjectOutputStream`'s reference cache before `writeObject(msg)` so updated `Team` purses are always fresh across laptops!

File 1 (Continued): `AuctionServer.java` — Cross-Network Exception Handling

📂 Stay in: AuctionServer.java (Scroll to Lines 224–236 & Lines 275–294)

```
// 1. Catching Rule Violations & Routing Error Back to ONLY That Client (Lines 275-294)
private void handleRemoteBid(NetworkMessage msg) {
    try {
        Team biddingTeam = teamService.getTeamById(bidTeamId);
        biddingTeam.setOwnerType(OwnerType.HUMAN);
        auctionService.placeHumanBid(biddingTeam, msg.getAmount());
    } catch (CricBidException ex) {
        send(new NetworkMessage(MessageType.BID_REJECTED).setText(ex.getMessage()));
    } catch (SQLException ex) {
        send(new NetworkMessage(MessageType.BID_REJECTED).setText("Database error: " +
ex.getMessage()));
    }
}

// 2. Graceful Laptop Disconnect Cleanup (Lines 224-235)
} catch (EOFException ignored) {
    // Client closed window normally
} catch (Exception e) {
    if (active) System.err.println("[AuctionServer] Client disconnected: " + e.getMessage());
} finally {
    clients.remove(this);
    closeQuietly();
    notifyClientListChanged(); // Updates live connected-laptop counter on Host!
}
```

Exact Words to Say to Sir:

“Sir, look at `handleRemoteBid()` on line 289: if Laptop 2 breaks an auction rule, `AuctionService` throws `CricBidException`. Our server catches it without crashing and calls `send(BID_REJECTED)` on ONLY Laptop 2’s socket, so Laptop 2 gets the warning popup while Laptop 1 and Laptop 3 continue undisturbed!”

File 2: `AuctionClient.java` & `AuctionSubject.java` — Auto-Retry `try-catch` & Swing EDT

📂 Open: src/main/java/com/cricbid/network/AuctionClient.java (Lines 80–125)

```
while (running) {
    try {
        Socket s = new Socket();
        s.connect(new InetSocketAddress(host, port), 3500);
        ...
        send(new NetworkMessage(MessageType.REGISTER_CLIENT)
                .setUsername(username).setTeamId(teamId).setText(teamName));

        while (running && !s.isClosed()) {
            Object obj = in.readObject();
            if (obj instanceof NetworkMessage) dispatchMessage((NetworkMessage) obj);
        }
    } catch (Exception e) {
        this.connected = false;
        notifyStatus(false, "Waiting for Auctioneer Host (retrying)...");
        try {
            Thread.sleep(2000); // Retries every 2 seconds instead of crashing!
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
            break;
        }
    } finally {
        closeSocketQuietly();
    }
}
```

- **Resilient Exception Handling:** If Laptop 2 logs in before Laptop 1 starts the server, `s.connect(...)` throws `IOException`. Instead of crashing, `catch` displays *"Waiting for Auctioneer Host (retrying)..."*, sleeps 2 seconds, and connects automatically as soon as Laptop 1 logs in!
- **Late-Join `SYNC_STATE` + `SwingUtilities.invokeLater`:** Immediately syncs the active player, countdown, and upcoming pool onto Swing's Event Dispatch Thread.

4. Viva Questions Sir Might Ask You (+ Exact Rubric Answers)

Q1: Which Java libraries and interfaces did you combine inside `com.cricbid.network`?

Answer: "For libraries, we used `java.net` (`ServerSocket`, `Socket`, `InetSocketAddress`), `java.io` (`ObjectOutputStream`, `ObjectInputStream`), and `java.util.concurrent.CopyOnWriteArrayList`. For interfaces, `AuctionServer` implements `AuctionObserver`, `ClientHandler` implements `Runnable`, and `NetworkMessage` implements `Serializable`."

Q2: Why do you catch `EOFException` separately when reading from `ObjectInputStream`?

Answer: "When a participant laptop closes its window, `readObject()` throws `java.io.EOFException` to indicate end-of-stream. Catching it cleanly lets our `finally` block remove that client and update Laptop 1's connected-laptops badge without an error crash."

Q3: Why create and `flush()` `ObjectOutputStream` BEFORE `ObjectInputStream`?

Answer: "`new ObjectInputStream()` blocks until it reads the stream header from the remote `ObjectOutputStream`. Initializing and flushing `ObjectOutputStream` first on both Server and Client prevents socket handshake deadlock."